

Task 5.2C - My DNS Server Draft

Introduction

For this task, after confirming with my tutor in the Monday 2PM workshop, I confirmed that the DNS query doesn't have to follow the actual real world formats. Therefore, I will be keeping it very straight simple and forward.

Pseudocode

Server

```
BEGIN
  Set the server address to 127.0.0.1
  Set the port to 5354
  Store the A and CNAME records for each domain and host (using a correct data type)

  Create an IPv4 UDP socket
  Bind the socket to the server address and port
  Display that the server is running

  WHILE the server is running
    Receive a query and the client's address
    Decode the query into text
    Split the query into a hostname and record type
    Convert the hostname to lowercase
    Convert the record type to uppercase

    FOR each domain in the DNS records
      IF the hostname matches the domain
        Use @ as the host label
      ELSE IF the hostname is a subdomain
        Extract the host label from the hostname
      END IF

      Search the host records for the requested record type
    END FOR

    IF a matching record is found
      Create a response containing the record type and value
    ELSE
      Set the response to "DNS record not found"
    END IF

    Encode the response
    Send the response to the client's address
  END WHILE
END
```

➔ Next page

Client

```
BEGIN
  Set the server address to 127.0.0.1
  Set the server port to 5354
  Create an IPv4 UDP socket
  Set a five-second timeout

  Ask the user to enter a hostname
  Ask the user to enter an A or CNAME record type
  Combine the hostname and record type with a comma
  Encode and send the query to the server

  IF a response is received within five seconds
  |   Decode and display the server's response
  ELSE
  |   Display that the server did not respond
  END IF

  Close the client socket
END
```

Python

The python source files for both the server and client (server.py and client.py) are attached to the end of this file.

I used the python sockets API with UDP (as that's what DNS usually uses) and used a simple dictionary to emulate a real world DNS record holding configuration with subdomains and multiple record type supports.

The server listens for incoming requests from the client, then checks for the hostname & record type (in the query) and then sends a response back to the client with the correct result or an error if that record doesn't exist.

Each main domain can contain multiple subdomains, and each subdomain can contain different record types like A records, CNAME records, etc... The @ system here is used to represent the root domain, while labels like www, mail, api represents subdomains here.

This is a really simple minimum viable implementation. It doesn't generate or parse real DNS packets, perform recursive DNS lookups, cache records, validate IP addresses, support record expiry, include proper security and error handling or any advanced features. The primary goal of this project is to demonstrate the main logical flow of DNS communication at a simpler, higher level, using UDP sockets.

Lessons learned: I didn't want to test around with port 53. Therefore, I tried port 5353 and apparently that port is always used by a Microsoft system service. Therefore, I changed the port to 5354 and tested with that configuration.

DNS Server & Client (Python Implementation)

```
1  # -----
2  # server.py
3  # -----
4
5  import socket
6
7  HOST = "127.0.0.1"
8  PORT = 5354
9
10 records = {
11     "example.com": {
12         "@": {
13             "A": "93.184.216.34"
14         },
15         "www": {
16             "CNAME": "@"
17         },
18         "mail": {
19             "A": "192.168.1.20"
20         },
21         "api": {
22             "A": "192.168.1.30"
23         },
24         "shop": {
25             "CNAME": "www"
26         }
27     },
28
29     "google.com": {
30         "@": {
31             "A": "142.250.70.14"
32         },
33         "www": {
34             "CNAME": "@"
35         },
36         "mail": {
37             "CNAME": "@"
38         }
39     }
40 }
41
42 server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
43 server_socket.bind((HOST, PORT))
44
45 print(f"DNS server is running on {HOST}:{PORT}")
46
47
48
49 while True:
50     data, client_address = server_socket.recvfrom(1024)
```

```

51
52     query = data.decode()
53     hostname, record_type = query.split(",")
54
55     hostname = hostname.lower().strip()
56     record_type = record_type.upper().strip()
57
58     print(f"Query received: {hostname} - {record_type}")
59
60     answer = None
61
62     for domain, domain_records in records.items():
63         if hostname == domain:
64             host_label = "@"
65         elif hostname.endswith(f".{domain}"):
66             host_label = hostname[:-(len(domain) + 1)] # exclude the . before the
domain
67         else:
68             continue
69
70         host_records = domain_records.get(host_label, {})
71         answer = host_records.get(record_type)
72         break
73
74     if answer is not None:
75         response = f"{record_type} record: {answer}"
76     else:
77         response = "DNS record not found"
78
79     server_socket.sendto(response.encode(), client_address)
80
81     # -----
82     # client.py
83     # -----
84     import socket
85
86     SERVER_HOST = "127.0.0.1"
87     SERVER_PORT = 5354
88
89     client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
90     client_socket.settimeout(5)
91
92     hostname = input("Enter hostname: ")
93     record_type = input("Enter record type (A or CNAME): ")
94
95     query = f"{hostname},{record_type}"
96
97     client_socket.sendto(
98         query.encode(),
99         (SERVER_HOST, SERVER_PORT)
100     )
101
102     try:

```

```
103     response, server_address = client_socket.recvfrom(1024)
104     print("Server response:", response.decode())
105
106 except socket.timeout:
107     print("The DNS server did not respond.")
108
109 client_socket.close()
110
111
112
113
```